Mutual TLS Client Certificate Authentication

with backup method, certificate reset and centralised revocation

Practical task - full assignment (option: Mutual TLS client certificates)

Information Technology Security Methods - VILNIUS TECH - 2026-2027 fall

Role	Team member	Main responsibility
Builder	Achraf YASMINE	PKI, nginx, backend, Docker Compose
Breaker	Ukachi	Security tests, attack scenarios, test log
Writer	José LOPES	STRIDE threat model, ASVS checklist, report

Repository: https://github.com/codeslayer918/ITSM-project

- Date: 9 November 2026

Contents

1. Introduction 3
2. Background 3
2.1 X.509 certificates and the chain of trust 3
2.2 Mutual TLS in TLS 1.3 3
2.3 Revocation 3
3. Architecture 4
4. PKI and chain of trust 5
5. Base mechanism: login and session 6
5.1 Certificate verification (two layers) 6
5.2 Session management 6
6. Backup authentication method 7
6.1 Second certificate (another device) 7
6.2 One-time backup codes 7
7. Credential reset (lost certificate) 8
8. Centralised revocation 9
9. Logging and monitoring 9
10. Threat model (STRIDE) of the login and recovery flows 10
11. Do the additions increase or decrease security? 12
12. OWASP ASVS 5.0 Level 1 checklist (V6, V7) 12
13. Security tests 13
13.1 Required cases and other defences 13
13.2 Key outputs 14
13.3 Attack scenarios 14
13.4 Findings 15
14. Reproducibility 15
15. Limitations and future work 16
16. Answers to the mini-questions 16
16.1 How does TLS 1.3 establish keys, and what does the certificate prove? 16
16.2 Persistent, time-limited and session cookies for secure authentication 16
16.3 Why is account recovery usually the weakest point? 16
16.4 What does MFA not protect against once the session cookie is stolen? 17
16.5 SIEM: why it matters, tools, authentication events to send 17
17. Team contributions 17
References 17

1. Introduction

The assignment asks for a simple web application whose user authentication relies on one mechanism, and - for the full grade - for three additions: a **backup authentication method**, a **credential reset** for users and a **centralised revocation** of user or group credentials by the infrastructure owner. The design must be justified with a threat model: what each addition lets an attacker try, and how it is blocked [14].

Our mechanism is **mutual TLS (mTLS) with X.509 client certificates**. During the TLS 1.3 handshake the server asks the browser for a certificate; the browser proves possession of the matching private key by signing the handshake transcript. No password exists anywhere in the system. On top of that base we built:

- **Backup method**: a second certificate on another device (like a second passkey) and 10 one-time backup codes hashed with Argon2id.
- **Reset**: a user who lost every certificate proves their identity with a backup code or a 15-minute magic link sent by e-mail, generates a new key pair locally and sends a CSR; the old certificates are revoked automatically, all sessions are closed and the user is notified.
- **Centralised revocation**: an administrator (role ADMIN and certificate with `OU=admins`) revokes one certificate, all certificates of a user, or all certificates of a group; revocation is enforced both by the application (immediately) and by nginx through a CRL (after an automatic reload).

The work was split between three roles, rotated so that everyone can explain every part during the live demonstration: Builder (application and mechanism), Breaker (tests and test log), Writer (threat model, ASVS checklist, report).

2. Background

2.1 X.509 certificates and the chain of trust

An X.509 certificate binds a public key to an identity (subject DN, alternative names) and is signed by a certification authority (CA) [3]. A relying party trusts a small set of **trust anchors** (root CAs) and accepts a certificate if it can build a chain of valid signatures from it to a trust anchor, with every certificate inside its validity period, every CA certificate allowed to sign (`basicConstraints CA:true`, `keyCertSign`) and the leaf allowed for the requested purpose (Key Usage and **Extended Key Usage**; `clientAuth` for TLS client authentication).

2.2 Mutual TLS in TLS 1.3

In TLS 1.3 [2] the server sends a `CertificateRequest` (with the list of acceptable CA names) inside the encrypted part of the handshake. The client answers with its `Certificate` and a `CertificateVerify` message: a signature over the transcript hash made with its private key. The server therefore learns that the peer (a) holds a certificate issued by a CA it trusts and (b) holds the private key **for this very handshake**, which prevents replay.

2.3 Revocation

A certificate stays cryptographically valid until it expires, so a compromised certificate must be **revoked**. The CA publishes a signed **Certificate Revocation List (CRL)** listing revoked serial numbers, with a `nextUpdate` date after which the CRL itself is stale [3]. OCSP is an online alternative; we use CRLs because nginx supports them natively for client certificates (`ssl_crl`) [12].

3. Architecture

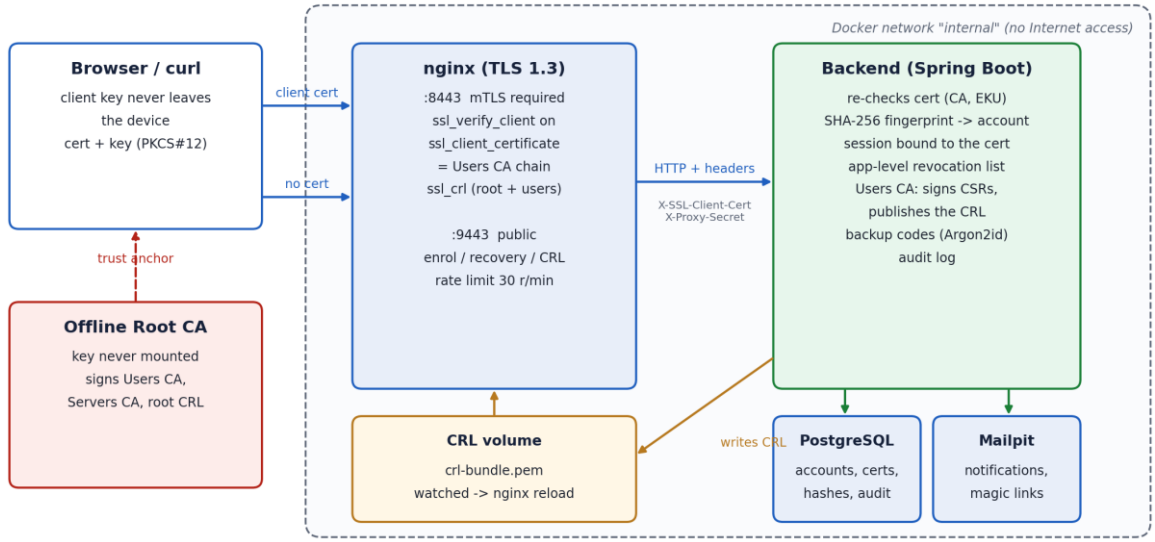


Figure 1 - Components and trust boundaries

Component	Technology	Role
PKI	OpenSSL 3 scripts (`pki/`)	Offline Root CA, intermediate **Users CA** (client certificates) and **Servers CA** (nginx certificate); EC P-384 CA keys encrypted with AES-256.
Reverse proxy	nginx 1.27 (`nginx/`)	Terminates **TLS 1.3**. Port 8443: `ssl_verify_client on`, trust = Users CA chain, `ssl_crl`. Port 9443: public, only enrolment, recovery and CRL. Forwards the certificate to the backend in headers.
Backend	Spring Boot 3.4 / Java 21 (`backend/`)	Re-validates the certificate, maps its SHA-256 fingerprint to an account, manages sessions, the application-level revocation list, the Users CA (CSR signing, CRL generation), backup codes, notifications and the audit log.
Database	PostgreSQL 16	Accounts, issued certificates, Argon2id hashes of backup codes, SHA-256 hashes of tokens, audit events, CRL number.
Mail	Mailpit	Fake SMTP server with a web UI: magic links and security notifications.
Orchestration	Docker Compose	Reproducible stack; backend and database on an **internal** network without Internet access; only nginx (8443, 9443) and Mailpit UI (localhost:8025) are published.

Trust boundaries. (1) Internet / nginx: TLS, client authentication. (2) nginx / backend: the backend trusts the `X-SSL-Client-*` headers only because nginx always overwrites them and adds a shared secret header; the backend port is not published. (3) backend / database and mail. (4) The offline Root CA, whose key is never mounted into any container.

4. PKI and chain of trust

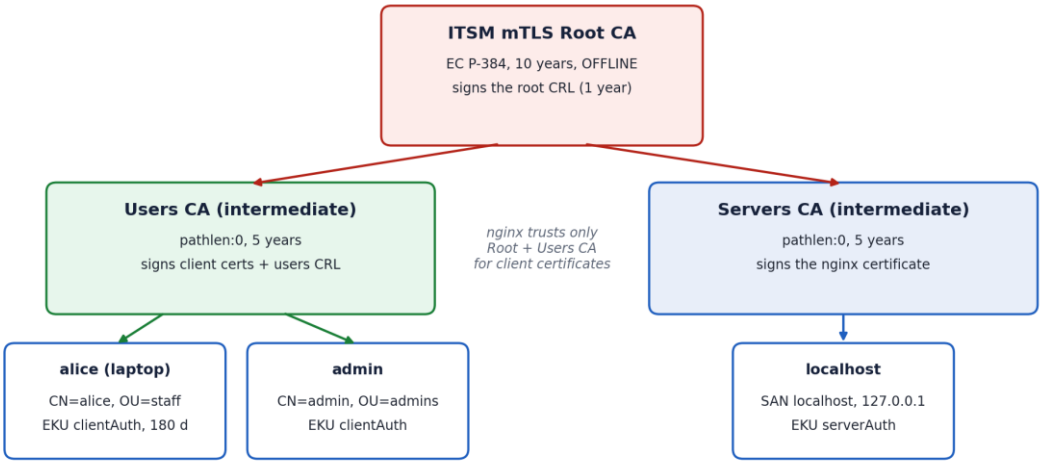


Figure 2 - Two-level PKI: offline root, one intermediate per purpose

The PKI is built by `pki/init-pki.sh` from a single OpenSSL configuration (`pki/conf/ca.cnf`). The Root CA signs the two intermediates and its own CRL, then goes offline: only the Users CA key is available to the backend (encrypted at rest, passphrase given as a Docker secret). The Servers CA only signs the nginx certificate, so a server certificate can never be accepted as a client certificate and the client trust store contains only `Users CA + Root`.

Certificate	Key	Validity	Important extensions
Root CA	EC P-384	10 years	CA:true, keyCertSign, cRLSign
Users CA / Servers CA	EC P-384	5 years	CA:true, **pathlen:0**, keyCertSign, cRLSign
nginx (localhost)	EC P-256	397 days	CA:false, digitalSignature, EKU serverAuth, SAN localhost / 127.0.0.1
Client (issued by the backend)	EC P-256/P-384 or RSA >= 2048 (user's choice)	180 days	CA:false, digitalSignature, **EKU clientAuth**, subject `CN=username, OU=group`, SAN e-mail, CRL distribution point, 127-bit random serial

Because nginx enables CRL checking for the **whole chain** (`X509_V_FLAG_CRL_CHECK_ALL`), the CRL file contains two CRLs: the root CRL (re-signed yearly in an offline ceremony, `pki/renew-root-crl.sh`) and the Users CA CRL (regenerated by the backend). Revoking the whole Users CA in the root CRL is the “revoke an entire CA” variant of group revocation.

5. Base mechanism: login and session

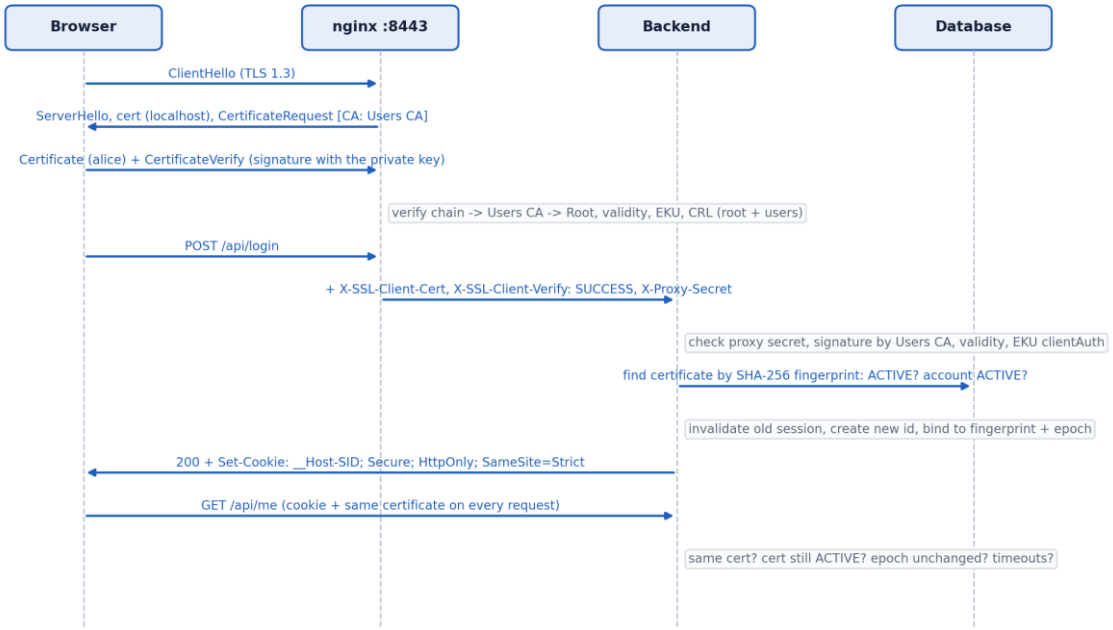


Figure 3 - Login sequence

5.1 Certificate verification (two layers)

1. **nginx** (TLS layer) verifies the chain up to the Root CA, the validity period, the purpose (EKU) and the CRLs. On failure it answers 403 with the OpenSSL reason (`error_page 495/496`); the request never reaches the backend.
2. nginx forwards the request with `X-SSL-Client-Verify`, `X-SSL-Client-Cert` (URL-encoded PEM), DN, serial, a listener marker and the shared proxy secret. `proxy_set_header` replaces any header of the same name sent by the client.
3. The **backend** re-checks the certificate (defence in depth): signature by the Users CA, validity period and presence of **EKU clientAuth** (OpenSSL accepts a certificate without any EKU; our backend does not - test T5b).
4. The SHA-256 fingerprint of the certificate is looked up in the `issued_certificate` table: unknown certificate -> 401, status REVOKED -> 403, disabled account -> 403.

5.2 Session management

- A successful `POST /api/login` invalidates any session presented by the client and creates a new one (**session fixation** defence). The session id is a 128-bit random value from Tomcat's `SecureRandom`.
- Cookie `__Host-SID`: `Secure`, `HttpOnly`, `SameSite=Strict`, `Path=/`, no `Domain`; cookie-only tracking.
- The session stores the account id, the **fingerprint of the certificate used to log in**, the account's `session_epoch` and the login time. On every protected request the filter checks that the same certificate is presented, that it is still ACTIVE in the database (application-level revocation list), that the epoch did not change (a reset or a revocation increments it and kills every session) and that the idle (15 min) and absolute (4 h) timeouts are respected.

- **Binding the session to the certificate** means that a stolen cookie is useless without the private key; when a cookie is presented with another certificate the session is treated as stolen and destroyed (test T11).
- CSRF: browsers send client certificates cross-site, so state-changing requests additionally require `Content-Type: application/json`, an allowed `Origin` and no `Sec-Fetch-Site: cross-site`.
- Administration requires **both** the database role ADMIN and a certificate with `OU=admins`; the OU is written by the CA, never by the user.

6. Backup authentication method

6.1 Second certificate (another device)

A logged-in user can enrol another device: the key pair is generated on that device, only the CSR is sent through the authenticated session (`POST /api/me/certificates`). At most three certificates can be active per account, each can be revoked individually, and the owner is notified by e-mail. This is the equivalent of registering a second passkey: losing one device does not require the recovery flow, the user simply revokes the lost certificate from the other device.

6.2 One-time backup codes

- 10 codes of 10 characters from a 31-symbol alphabet (about **49.5 bits** each), generated with `SecureRandom` and shown once (at first enrolment or on demand).
- Stored as **Argon2id** hashes (m = 19 MiB, t = 2, p = 1, 16-byte salt) as recommended by OWASP [6][11].
- **Single use** through an atomic `UPDATE ... WHERE used_at IS NULL`: two parallel requests with the same code produce exactly one success (attack A7).
- A backup code **never logs in**: it can only be used to enrol a new certificate through the reset flow, which revokes every existing certificate and notifies the user, so a misuse is loud.
- The verification does the same Argon2 work (10 hashes) whether the username exists or not, so the response time does not reveal valid usernames (attack A9: 384 ms vs 361 ms).

7. Credential reset (lost certificate)

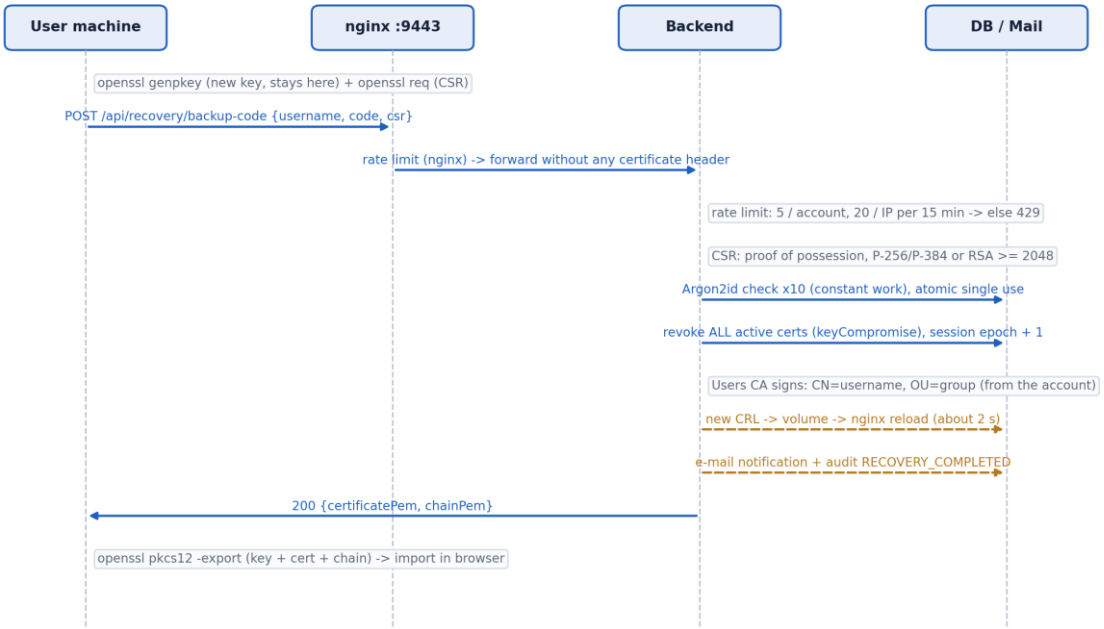


Figure 4 - Reset with a backup code (the magic-link variant only differs in the proof of identity)

The reset is served on the **public listener** (port 9443, no client certificate) and offers two proofs of identity:

- **Username + backup code** (available to everybody, the only option for administrators).
- **Magic link** sent to the registered e-mail: 256-bit random token, stored as SHA-256, valid **15 minutes**, **single use**, placed in the URL **fragment** (`#token=...`) so that it is never sent to the server in a URL, never logged and never leaked through `Referer`. The request always returns the same `202` message and all the work (lookup, token, e-mail) is done asynchronously, so neither the body nor the timing reveals whether the address exists. Magic links are **disabled for administrators**: taking over an administrator's mailbox must not be enough to obtain an admin certificate.

In both cases the user generates the key pair **locally** (an OpenSSL one-liner or `scripts/recover.sh`) and sends only a **CSR**. The backend checks the proof of possession (CSR signature), accepts only EC P-256/P-384 or RSA >= 2048, refuses a key that was already certified, ignores the requested subject and extensions, and signs a certificate built from the account. Then it **revokes every previously active certificate** (reason keyCompromise), increments the session epoch (all sessions die), regenerates the CRL and **notifies** the user. The server never sees a private key: we deliberately did not implement server-side PKCS#12 generation, which would give the server (and anyone who compromises it) every user's key.

Rate limiting: 5 attempts per account and 20 per IP address in a sliding 15-minute window, 3 magic links per e-mail per hour, plus an nginx limit of 30 requests/min per IP on the public API. Excess requests get `429` with `Retry-After`. A disabled account cannot be recovered, even with a valid backup code.

8. Centralised revocation

Scope	Endpoint	Effect
One certificate	`POST /api/admin/certificates/{serial}/revoke`	Status REVOKED + reason; sessions of the account invalidated; CRL regenerated.
One user	`POST /api/admin/users/{u}/revoke}disableAccount`	All active certificates revoked; optionally the account is disabled, which also blocks recovery and invalidates pending tokens.
One group	`POST /api/admin/groups/{g}/revoke`	All active certificates whose OU is the group are revoked, member sessions closed; optional disabling. The `admins` group cannot be revoked as a whole (lock-out protection).
Whole CA	`pki/renew-root-crl.sh users` (offline)	The Users CA itself is revoked in the root CRL: every client certificate stops working.

Two enforcement layers. The application checks the certificate status in the database on **every** request, so a revocation is effective immediately. At the same time the backend regenerates the signed CRL (monotonic CRL number, `nextUpdate` = 24 h) and writes the bundle atomically to a volume shared with nginx; a watcher in the nginx container detects the change and runs `nginx -s reload` within about 2 seconds. Test T4 shows both layers: first `403 certificate_revoked` from the application, then `403 FAILED:certificate revoked` from nginx. TLS session resumption is disabled so that every connection is verified against the current CRL.

Fail closed. If the CRL is not refreshed, nginx rejects every client certificate once `nextUpdate` has passed (attack A14 shows `FAILED:CRL has expired`). This is a deliberate choice (a stale CRL could hide a revocation); the backend republishes the CRL every 10 minutes and at start-up to keep it fresh.

9. Logging and monitoring

Every security event is written to the `audit_event` table (shown on the admin page) and to the `AUDIT` logger on stdout, in a key=value format ready to be shipped to a SIEM: login success/failure, logout, rejected session (cookie with another certificate, revoked certificate), enrolment, certificate issued/revoked, user and group revocation, backup codes generated/used, recovery requested/completed/failed, rate limiting, denied admin access and CRL publication. Each event carries the time, user, source IP, user agent, certificate serial and outcome. **Secrets are never logged** (no codes, tokens, session ids or keys), and user-controlled values are sanitised against log injection. Users receive an e-mail for every certificate issuance, reset and revocation, which is the main way to detect an account takeover.

10. Threat model (STRIDE) of the login and recovery flows

Assets: accounts (especially administrators), the Users CA private key, client private keys, session cookies, backup codes and magic links, the CRL and the audit log. **Attackers**: (A) remote without credentials, (B) holding one stolen credential (device, cookie, code sheet, mailbox), (C) malicious regular user, (D) attacker on the backend host. Evidence = test ids of section 13. Residual risk: L low, M medium, H high.

#	STRIDE	Flow	Threat	Mitigation	Evidence	Res.
S1	Spoofing	Login	Stolen certificate + private key (device theft, malware)	Key generated on the device; revocation by owner (other device), admin or reset: immediate in the app, TLS level within ~2 s; 180-day validity	T4, T13, A11	M
S2	Spoofing	Login	Certificate from another CA with an identical DN	nginx trusts only the Users CA chain; backend verifies the signature with the Users CA key	T2	L
S3	Spoofing	Login	Inject X-SSL-Client-* headers / bypass nginx	Headers always overwritten (deleted on the public port); proxy secret; backend port not published	T14, A2, A3	L
S4	Spoofing	Session	Replay a stolen session cookie	HttpOnly, Secure, __Host-; session bound to the certificate fingerprint, destroyed on mismatch	T11	L
S5	Spoofing	Session	Session fixation	New id at every login; presented ids never adopted	T9	L
S6	Spoofing	Recovery	Mailbox takeover, then magic link	Single use, 15 min, notification, all old certificates revoked (loud), disabled for admins, rate limited	T7, A13	M
T1	Tampering	Enrolment	CSR asking for CN=admin, OU=admins, CA:true	CA ignores CSR subject/extensions; builds them from the account	A1	L
T2	Tampering	Revocation	Replace / freeze the CRL to revive a revoked cert	Signed CRL, atomic write, fail closed on expiry, independent app-level list	T4, A14	L
T3	Tampering	Enrolment	Weak key, no proof of possession, compromised key reused	Key type/size policy, CSR signature check, a key is certified only once	A5, A6	L
T4	Tampering	Session	CSRF (client certificate sent cross-site)	SameSite=Strict, Origin / Sec-Fetch-Site checks, JSON-only bodies	T15	L
R1	Repudiation	All	Deny a reset, revocation or login	Audit trail (time, user, IP, UA, serial) in DB + SIEM-ready log; e-mail notifications	T17, A15	M
I1	Info disclosure	Recovery	Account enumeration (message or timing)	Generic messages, constant Argon2 work, asynchronous magic-link processing (F2 fixed)	T7, A9	L
I2	Info disclosure	Recovery	Steal codes / tokens from DB, logs, URLs	Argon2id / SHA-256 at rest, token in URL fragment, secrets never logged	A15	L
D1	Denial of service	Recovery	Brute force, Argon2 CPU exhaustion	Per-account, per-IP, per-e-mail and nginx limits, 429 + Retry-After	T8	L

#	STRIDE	Flow	Threat	Mitigation	Evidence	Res.
D2	Denial of service	Login	Stale CRL blocks every login	CRL refreshed every 10 min and on revocation (valid 24 h); fail closed by design	A14	M
D3	Denial of service	Revocation	Lock every administrator out	Revocation of the admins group refused; audit	A12	L
E1	Elevation	Recovery	Abuse the reset flow (guess or race a code, disabled account)	49.5-bit codes, Argon2id, atomic single use, rate limits, disabled accounts excluded, notification	T6, T8, A7, A8	L
E2	Elevation	Login	Regular user becomes administrator	Role ADMIN **and** OU=admins written by the CA; no e-mail recovery for admins	T12, A1, A13	L
E3	Elevation	Enrolment	Replay an invitation; act on another user's certificate (IDOR)	Single-use 24 h tokens; ownership checks	A4, A10	L
E4	Elevation	PKI	Compromise the backend host and its Users CA key	Key encrypted at rest, root offline, backend not exposed, root can revoke the Users CA	design	H

The highest residual risks are **E4** (the issuing CA is online on the backend host), **S1** (a stolen device remains usable until it is revoked), **S6** (e-mail is a recovery factor for non-admin users) and **D2** (availability depends on CRL freshness).

11. Do the additions increase or decrease security?

Each addition creates a new way to obtain or remove access. We list what it lets an attacker try, how we blocked it and the remaining effect, using the threat model rather than opinion.

Addition	What it lets an attacker try	How we blocked it	Net effect
Second certificate	More private keys exist, so more devices can be stolen (S1); a session thief could try to add his own device	Only from an authenticated session bound to a certificate (a stolen cookie is not enough, T11); max 3 active; owner notified; each certificate revocable from the other device	**Slightly larger attack surface, much better availability**: losing one device no longer forces the weaker recovery path
Backup codes	Guess a code (E1), steal the code sheet, race the same code (A7), learn which usernames exist (I1)	49.5 bits per code, Argon2id, atomic single use, 5 attempts / 15 min per account, generic errors and constant work, codes cannot log in, use is loud (all certificates revoked + e-mail)	Neutral to positive: an offline factor that replaces a help-desk procedure; guessing is infeasible (10 codes x 2^-49.5 per attempt)
Reset by backup code	Turn a stolen code sheet into a new certificate	Rate limits; disabled accounts excluded; every old certificate revoked and the owner notified, so the takeover is visible and the legitimate user loses access immediately (and reports it)	**Lowers** the account's security to min(certificate, code sheet) - acceptable because the sheet is offline, single use and the reset is loud
Reset by magic link	Mailbox takeover = certificate (S6); enumeration of addresses (I1); replay of a link (E3)	Single use, 15 min, SHA-256 at rest, fragment-only, identical answer and timing (F2 fixed), notification, rate limits; **not available for administrators**	**Lowers** security for regular users to the security of their mailbox; accepted for usability, explicitly excluded for admins
Centralised revocation	Abuse by a malicious admin (mass lock-out, D3); stale or forged CRL (T2, D2)	Admin = role + OU=admins certificate; admins group cannot be revoked at once; every action audited; CRL signed, refreshed, fail closed; app-level list independent of nginx	**Increases security**: shortens the useful life of a stolen certificate from up to 180 days to seconds; the cost is a new availability dependency (CRL freshness)

Conclusion. Formally, every recovery path lowers the strength of an account to the weakest accepted proof: for regular users min(certificate, backup code, mailbox), for administrators min(certificate, backup code). Without these paths, however, a lost device would be handled by an ad-hoc help-desk procedure, which is the classic weakest point (social engineering). By making recovery explicit, rate limited, single use, loud (revocation of everything + notification) and by excluding e-mail for administrators, we keep the weakest link under control. Revocation clearly raises security by turning a long-lived credential into one that can be cancelled within seconds. Overall the full solution **increases** the practical security of the system, at the price of three accepted residual risks (S6, D2, E4) documented above.

12. OWASP ASVS 5.0 Level 1 checklist (V6, V7)

Requirement ids are those of ASVS 5.0.0 [1]; texts are paraphrased. **Result: 11 applicable Level 1 requirements, 10 met, 1 not met (6.3.2), 8 not applicable (password requirements).**

ID	Requirement	Status	Justification / evidence
6.1.1	Document rate limiting / anti-automation and how they avoid malicious lockout	Met	Section 7 and README; temporary sliding windows that only block recovery, never login (residual: an attacker can delay a victim's recovery by 15 min).
6.2.1-6.2.8	Password policy (length, change, breached list, masking, paste, no truncation)	N/A	No passwords: X.509 certificates. Backup codes are lookup secrets and cannot log in.

ID	Requirement	Status	Justification / evidence
6.3.1	Controls against credential stuffing / brute force implemented as documented	Met	Application and nginx rate limits (T8, A7, A9).
6.3.2	Default accounts (root, admin, sa) absent or disabled	Not met	The bootstrap account is named 'admin'. It has no default credential (random single-use 24 h token + certificate). Fix: configure a personal admin username before the first start.
6.4.1	Activation codes random, short-lived or single use	Met	256-bit enrolment tokens (24 h) and magic links (15 min), single use, SHA-256 at rest (T7, A4).
6.4.2	No password hints or secret questions	Met	None exist.
7.2.1	Session tokens verified by a trusted backend	Met	Server-side session checked by SessionAuthFilter.
7.2.2	Dynamically generated tokens, not static API keys	Met	Random session id created at login.
7.2.3	Reference tokens unique, CSPRNG, >= 128 bits	Met	Tomcat SecureRandom, 16 bytes (32 hex characters, T9).
7.2.4	New session token on authentication, old one terminated	Met	T9: id changes at every login, old id dead, attacker id not adopted.
7.4.1	No further use after logout / expiry	Met	Server-side invalidation; idle 15 min / absolute 4 h; A11.
7.4.2	All sessions terminated when the account is disabled	Met	Epoch increment + certificate revocation; A11, T13.

Beyond Level 1 (information): met - 6.4.3 (reset never bypasses the certificate), 6.5.1-6.5.4 (backup codes as lookup secrets), 7.3.1/7.3.2 (idle and absolute timeouts, not automatically tested), 7.4.3 (sessions terminated after a reset), 3.3.1-3.3.4 (cookie attributes), 3.4.1 (HSTS one year), 3.4.3 (CSP with object-src 'none', after fix F1), 12.1.1 (TLS 1.3 only). Not met - 6.3.3 (the certificate is a single factor; a PIN-protected smart card would make it MFA), 7.5.1 (no re-authentication before sensitive actions), 7.5.2 (no list of active sessions); partially - 7.4.5 (admins end sessions only through revocation / disabling).

13. Security tests

The Breaker wrote two harnesses that run against the real stack and keep the raw output: `tests/security-tests.sh` (required cases, **35/35 pass**) and `tests/attack-scenarios.sh` (adversarial scenarios, **29/29 pass**). The full outputs are in `tests/logs/` and commented in `docs/TEST-LOG.md`. The negative-test certificates (other CA, expired, wrong EKU, no EKU) are produced by `pki/make-test-certs.sh`.

13.1 Required cases and other defences

ID	Case	Observed result
T1	No client certificate	403 client_certificate_required
T2	Certificate from another CA (identical DN)	403 FAILED:unable to verify the first certificate
T3	Expired certificate	403 FAILED:certificate has expired
T4	Revoked certificate	403 certificate_revoked (app), then 403 FAILED:certificate revoked (nginx)
T5	Wrong EKU (emailProtection)	403 FAILED:unsuitable certificate purpose
T5b	No EKU at all	403 client_certificate_wrong_eku (backend)
T6	Reset with a backup code	200; old session dead; code single use; e-mail sent
T7	Reset with a magic link	same 202 for known / unknown; works once; none for admin

ID	Case	Observed result
T8	Rate limit on reset	5 x 401, then 429 + Retry-After: 896
T9	Session regenerated at login	new id each login; old id 401; attacker id not adopted
T10	Cookie flags	__Host-SID; Path=/; Secure; HttpOnly; SameSite=Strict
T11	Stolen cookie + other certificate	401 session_certificate_mismatch
T12	Non-admin on admin API	403 admin_required
T13	Group revocation	sessions 403 immediately; TLS refusal after reload
T14	Header spoofing, exposure, TLS 1.2	404; port closed; alert protocol version; TLS 1.3 only
T15	CSRF	403 bad_origin; 415 for text/plain
T16	Second device	200, same account

13.2 Key outputs

Revoked certificate (T4) - the application refuses it immediately, nginx after the CRL reload:

```
$ mtls clients/test-144514/revoked -X POST https://localhost:8443/api/login ...
HTTP/2 403
{"error":"certificate_revoked","message":"This certificate has been revoked."}
nginx-1  | [crl-watch] 2026-11-09T12:45:38Z CRL changed, nginx reloaded
    Serial Number: 2B8565CDFA2AE8768698126BEC2156A1   (CRL Number 36)
$ mtls clients/test-144514/revoked -X POST https://localhost:8443/api/login ...
HTTP/2 403
{"error":"client_certificate_rejected","reason":"FAILED:certificate revoked"}
```

Rate limiting on the reset endpoint (T8):

```
attempt 5: HTTP/2 401   {"error":"invalid_credentials","message":"Invalid username or backup
code."}
attempt 6: HTTP/2 429  retry-after: 896  {"error":"rate_limited","message":"Too many attempts, try
again later."}
```

Session regeneration and cookie flags (T9, T10):

```
set-cookie: __Host-SID=EDAB2F1A704C10CE65AEA18BE9228871; Path=/; Secure; HttpOnly; SameSite=Strict
set-cookie: __Host-SID=C61C30D4516D0E425067E130766D1A96; Path=/; Secure; HttpOnly; SameSite=Strict
$ ... -H 'Cookie: __Host-SID=EDAB2F1A704C10CE65AEA18BE9228871' https://localhost:8443/api/me
HTTP/2 401
{"error":"not_authenticated","message":"No session: POST /api/login first."}
```

Expired and wrong-purpose certificates (T3, T5) with `openssl s_client` and curl:

```
$ printf 'GET / HTTP/1.0\r\n\r\n' | openssl s_client -connect localhost:8443 -cert expired.crt -key
expired.key -quiet
{"error":"client_certificate_rejected","reason":"FAILED:certificate has expired"}
$ curl --cert wrong-eku.crt --key wrong-eku.key -X POST https://localhost:8443/api/login ...
{"error":"client_certificate_rejected","reason":"FAILED:unsuitable certificate purpose"}
```

13.3 Attack scenarios

ID	Attack	Observed result
A1	CSR requesting CN=admin, OU=admins, CA:true	issued as CN=mallory, OU=staff, CA:FALSE; admin 403
A2	Header injection on 8443	headers overwritten, logged in as the real certificate owner
A3	Direct access to the backend	403 without proxy secret; port not published
A4	Replay of an enrolment token	401 invalid_token
A5	Re-certify an already used key	409 key_reuse
A6	RSA-1024 CSR / corrupted CSR signature	400 (key size) / 400 (proof of possession)
A7	Same backup code twice in parallel	exactly one success

ID	Attack	Observed result
A8	Valid code of a disabled account	401 invalid_credentials
A9	Timing enumeration (codes / magic link)	384 vs 361 ms; 13 vs 13 ms (after F2)
A10	IDOR on another user's certificate	404, certificate still active
A11	Logout / disabled account sessions	401 / 403
A12	Revoke the admins group	409 self_lockout
A13	Magic link for the admin address	generic 202, no e-mail
A14	Expired CRL pushed to nginx	403 FAILED:CRL has expired (fail closed), restored
A15	Audit trail / secrets in logs	all events present; no code in logs

13.4 Findings

ID	Severity	Finding	Status
F1	Low	CSP without object-src 'none' (ASVS 3.4.3)	Fixed in the nginx configuration
F2	Medium	E-mail enumeration by timing on the magic-link request (about 43 ms vs 20 ms)	Fixed: asynchronous MagicLinkWorker, re-test 13 ms vs 13 ms
F3	Info	nginx returns the OpenSSL verification reason in the 403 body	Accepted for the demo; generic message in production
F4	Low	Rate-limit counters and sessions in memory (reset on restart, single instance)	Accepted; shared store in production
F5	Low	Repeated bad codes can delay a victim's recovery by 15 min	Accepted, documented (ASVS 6.1.1)
F7	High (design)	Users CA key online on the backend host	Accepted for the lab; HSM / separate signing service

Not tested automatically because of the waiting time: idle (15 min) and absolute (4 h) session timeouts, expiry of magic links (15 min) and invitations (24 h).

14. Reproducibility

Requirements: Docker with Compose v2, bash, OpenSSL, curl and Python 3 (we used Windows 11 + WSL2 Ubuntu, Docker 29.5, OpenSSL 3.0.2). All versions are pinned in the Dockerfiles and the Maven POM (Spring Boot 3.4.5, BouncyCastle 1.80, nginx 1.27, PostgreSQL 16, Mailpit 1.21).

```
./scripts/setup.sh                      # PKI, negative-test certificates, .env with random secrets
docker compose up -d --build            # db, mail, backend (unit tests run during the build),
nginx
./scripts/enroll.sh --bootstrap-admin   # first administrator: local key + CSR -> certificate,
PKCS#12, backup codes
./tests/security-tests.sh               # required cases   -> tests/logs/security-test-*.log
./tests/attack-scenarios.sh             # attack scenarios -> tests/logs/attack-scenarios-*.log
```

Browser demonstration: trust `pki/out/root-ca/ca.crt` as a root authority, import `clients/admin/client.p12`, open https://localhost:8443. Repository layout: `pki/` (PKI scripts), `nginx/`, `backend/`, `scripts/` (setup and client-side scripts), `tests/`, `docs/` (threat model, ASVS checklist, test log, this report). Generated secrets (`.env`, `pki/out/`, `clients/`) are excluded from Git.

15. Limitations and future work

- Move the Users CA key to an HSM or an isolated signing service (residual risk E4); automate the offline root ceremony.
- Make the certificate a real second factor: non-exportable keys in a TPM / smart card with a PIN.
- Shared rate-limit and session store (Redis or database) for several backend instances; OCSP stapling as a complement to CRLs.
- Re-authentication before sensitive actions and a list of active sessions (ASVS 7.5.1 / 7.5.2); configurable admin username (6.3.2).
- Ship the AUDIT log to a SIEM with alert rules (section 16.5) and write-once storage.

16. Answers to the mini-questions

16.1 How does TLS 1.3 establish keys, and what does the certificate prove?

The client sends a `ClientHello` with its supported cipher suites and an ephemeral (EC)DHE public key (`key_share`, e.g. X25519). The server answers with a `ServerHello` containing its own ephemeral share; both sides compute the same shared secret and feed it into the HKDF-based key schedule (early secret, handshake secret, master secret) to derive the handshake and then the application traffic keys [2]. Everything after the `ServerHello` is encrypted, static RSA key transport is removed, and ephemeral keys give **forward secrecy**. The server then sends its `Certificate` and a `CertificateVerify` (signature over the transcript) and both sides send `Finished` MACs.

A certificate proves that a CA trusted by the verifier bound **this public key** to **this identity** for **these purposes** (KU/EKU) during **this period**, and - if revocation is checked - that the binding has not been withdrawn. Combined with `CertificateVerify`, the peer also proves that it holds the private key **now**. It does not prove that the person at the keyboard is the legitimate owner, that the device is not compromised, or what the holder is authorised to do; in our system authorisation comes from the account and the OU set by our own CA.

16.2 Persistent, time-limited and session cookies for secure authentication

A **session cookie** has no `Expires`/`Max-Age` and is discarded when the browser session ends; a **persistent cookie** survives restarts until its expiry date (“remember me”) and therefore gives a stolen cookie a long useful life; a **time-limited** session is one whose validity is enforced **on the server** (idle and absolute timeouts), independently of what the browser keeps. For authentication the safe choice is a session cookie that carries only an opaque random id, with server-side idle and absolute timeouts and the `Secure`, `HttpOnly`, `SameSite` attributes and `__Host-` prefix [7][10]. Persistent login tokens should be separate, revocable, and should not allow sensitive actions without re-authentication. We use a session cookie with 15-minute idle and 4-hour absolute limits, bound to the client certificate.

16.3 Why is account recovery usually the weakest point?

Recovery is designed for people who have **lost** their normal credential, so by construction it asks for less proof; it is an alternative authentication path, and an account is only as strong as its weakest path. It relies on other channels with their own weaknesses (mailbox takeover, SIM swap for SMS, help-desk social engineering that bypasses strong MFA), it is used rarely and is therefore less tested and monitored, and usability pressure pushes towards lenient procedures [8][14]. Our answers: single-use, short-lived, rate-limited proofs; identical answers whether the account exists or not; revocation of all credentials and notification after a reset; no e-mail recovery for administrators.

16.4 What does MFA not protect against once the session cookie is stolen?

MFA protects the **authentication event** only. The session cookie issued afterwards is a bearer token: whoever presents it is treated as the user until the session ends. Info-stealer malware, XSS on a non-HttpOnly cookie or adversary-in-the-middle phishing proxies (which relay the MFA prompt and capture the resulting cookie) therefore bypass MFA completely, and every action inside the session is allowed. Countermeasures are binding the session to the client (mTLS-bound tokens [15], DPoP, device-bound session credentials), short sessions, re-authentication for sensitive actions and anomaly detection. In our design the session is bound to the TLS client certificate: a stolen cookie presented without the matching private key is rejected and the session destroyed (test T11).

16.5 SIEM: why it matters, tools, authentication events to send

A Security Information and Event Management system centralises logs from all components, keeps them out of reach of an attacker who compromises one host, **correlates** events across sources (e.g. many failures over many accounts = credential stuffing; logins from two distant countries within an hour), raises alerts, supports forensic investigation and satisfies retention / breach-notification obligations [9][13]. Typical tools: Splunk, Elastic Security (ELK), Microsoft Sentinel, IBM QRadar, Wazuh or Graylog, fed by collectors such as Filebeat, Fluent Bit or syslog.

Authentication events to send: login success / failure (user, IP, user agent, time), logout and session revocation, MFA / credential enrolment and removal, recovery requested / completed / failed, credential (certificate) issued and revoked, privilege changes and admin actions, rate limiting and lockouts, denied access. Never send passwords, codes, tokens or session ids. Example rules for our system: more than 5 `RECOVERY_FAILED` for one account; `RECOVERY_COMPLETED` followed by `CERTIFICATE_ISSUED via=ADDITIONAL_DEVICE` within minutes; any `SESSION_REJECTED` “different client certificate” (cookie theft); `GROUP_REVOKED` outside a change window; `ADMIN_ACCESS_DENIED`.

17. Team contributions

Role	Deliverables	Where
Builder	OpenSSL PKI (offline root, Users / Servers CA); nginx TLS 1.3 + mTLS + CRL auto-reload; Spring Boot backend (login, session binding, backup codes, reset, revocation, CRL publication, audit); minimal frontend; Docker Compose; client scripts; fixes F1 and F2	pki/, nginx/, backend/, scripts/, docker-compose.yml, BuilderPart.md
Breaker	Negative-test certificates; test harness for the required cases; adversarial scenarios (race, timing, header injection, stale CRL, IDOR...); test log and findings	tests/, docs/TEST-LOG.md, BreakerPart.md
Writer	STRIDE threat model; ASVS 5.0 L1 checklist; answer to “do the additions raise or lower security”; mini-questions; this report and its figures	docs/THREAT-MODEL.md, docs/ASVS-CHECKLIST.md, docs/report/, WriterPart.md

Each member committed their own part to the Git repository; roles were rotated during the preparation so that everyone can explain every part in the live demonstration.

References

[1] OWASP Foundation, Application Security Verification Standard 5.0.0, May 2025. https://owasp.org/www-project-application-security-verification-standard/

[2] E. Rescorla, RFC 8446 - The Transport Layer Security (TLS) Protocol Version 1.3, IETF, 2018.

[3] D. Cooper et al., RFC 5280 - Internet X.509 Public Key Infrastructure Certificate and CRL Profile, IETF, 2008.

[4] M. Nystrom, B. Kaliski, RFC 2986 - PKCS #10: Certification Request Syntax Specification v1.7, IETF, 2000.

[5] NIST, SP 800-63B-4 - Digital Identity Guidelines: Authentication and Authenticator Management, 2025.

[6] OWASP Cheat Sheet Series, Password Storage Cheat Sheet (Argon2id parameters).

[7] OWASP Cheat Sheet Series, Session Management Cheat Sheet.

[8] OWASP Cheat Sheet Series, Forgot Password Cheat Sheet.

[9] OWASP Cheat Sheet Series, Logging Cheat Sheet.

[10] A. Barth, RFC 6265 - HTTP State Management Mechanism, IETF, 2011, and draft-ietf-httpbis-rfc6265bis (SameSite, cookie prefixes).

[11] A. Biryukov et al., RFC 9106 - Argon2 Memory-Hard Function for Password Hashing, IETF, 2021.

[12] nginx documentation, Module ngx_http_ssl_module.
https://nginx.org/en/docs/http/ngx_http_ssl_module.html

[13] OWASP Top 10:2025 (A07 Authentication Failures, A09 Security Logging and Alerting Failures).

[14] Information Technology Security Methods, course slides ITSM_2PL, VILNIUS TECH, 2026-2027.

[15] B. Campbell et al., RFC 8705 - OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens, IETF, 2020.

[16] A. Shostack, Threat Modeling: Designing for Security, Wiley, 2014 (STRIDE).